Reducing the Cost of LLM Coding and Reasoning Models with Routers

Anonymous authors
Paper under double-blind review

Abstract

Routers typically choose a model per query by trading predicted correctness against cost. However, most price each model at a fixed cost, assuming that the number of tokens each model outputs does not vary significantly. We find that the output length of reasoning models varies by an order of magnitude across queries, and ask when per-query cost is worth predicting, what a prediction captures, and how cheaply it can be had. We predict **both** success probability **and** route-specific output length from the prefill activations of a single small frozen 4B LLM, using a linear readout per route. On 6,500 MMLU-Pro, 1,000 Omni-MATH, and 341 LiveCodeBench tasks, predicted costs reduce spending at matched accuracy by 28.8%, 34.8% and 33.7% relative to median-length pricing. Perfect per-query cost knowledge would save 45–60%, so our readouts capture half to three quarters of the achievable saving, and they beat every alternative estimator we evaluated. Across nine pools the saving follows two conditions: output length must vary within a route more than prices differ between routes, which holds for reasoning models but much less for non-reasoning models on the same problems, and that variation must be legible from the prompt. Pricing from difficulty alone matches the readouts where length tracks difficulty. The dedicated readout is what generalises: it keeps most of its saving with 20 labelled problems, when a new model is added from 10 examples, and when a router trained on other benchmarks is applied to a new one without labels, where text-embedding and item-response estimators fall well behind it. Every small LLM prefill we tried (0.6–8B, four model families) can be used as a router in this way.

1 Introduction

Routing reasoning models requires predicting both correctness and cost before generation. Token rates are published; the number of tokens is not. A model that is cheap on average can be expensive on a particular problem: on LiveCodeBench, a single model's output length differs 11–121× between its 10th- and 90th-percentile problems (Fig. 1).

Prefill routers predict correctness from a small encoder's prompt activations; the prefill router of Varshney et al. (Varshney et al., 2026) explicitly prices outputs at median training length. We read output length from the same activations (Fig. 2). Cost prediction adds one linear readout per route to an existing prefill router, without another encoder pass.

This raises three questions, which organise the paper. When is per-query cost worth predicting? What does a prediction of it capture? And how cheaply can a router obtain it: with how many labels, for how many new models, and on how many new benchmarks?

We contribute: (i) a measurement of what per-query cost knowledge is worth on nine pools, under one protocol at billed prices (29–60% of spending at matched accuracy wherever reasoning lengths vary), and on the same problems with non-reasoning routes, where it is worth much less and is largely not legible from the prompt (Section 5); (ii) prefill cost readouts that reduce spending by 28.8–34.8% relative to median pricing on three large test sets and outperform mean-length, prompt-feature, text-embedding and ZeroRouter cost estimators (Table 1); (iii) a mechanism: per-query length is mostly a problem-level property shared across

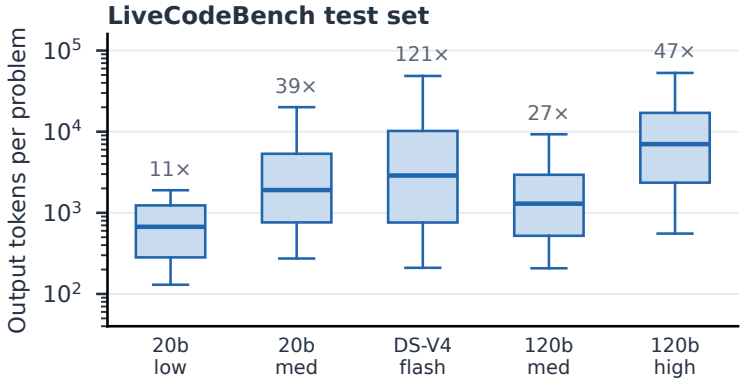


Figure 1: **Output length varies by orders of magnitude within one model.** Per-problem output tokens by route on the LiveCodeBench test set (log scale). Boxes: interquartile range; whiskers: 5th–95th percentile; line: median; above each box: 90th/10th-percentile ratio.

routes; where it tracks difficulty, pricing from the success readouts alone is enough, and where it tracks the work a question requires, only a dedicated readout captures it (Section 6). The two failures we find have identifiable causes: too little headroom, or headroom that no estimator can read from the prompt (Section 8); (iv) generalisation: the dedicated readout keeps most of its saving with few labels, when a new model is onboarded from a handful of examples, and when a router trained on other benchmarks is applied to a new one without labels; the external estimators do not, and pricing from difficulty transfers only between benchmarks whose difficulty means the same kind of work (Section 7); (v) controls showing the gain is the prefill representation: it holds for every small prefill we tried (0.6–8B, four families), and our linear readouts on fixed layers match the prefill router's full success pipeline on four of five sets. As a side note, the same model's provider endpoints drift; the readouts can absorb this with one offset per endpoint, and we pin providers instead.

Related work. Learned routers pick one model per query from the prompt, trading predicted quality against price: RouteLLM learns a strong-versus-weak decision from preference data (Ong et al., 2024), RouterBench standardizes the evaluation (Hu et al., 2024), and IRT-Router models query difficulty and model ability with item response theory (Song et al., 2025). Closest to us, the prefill router of Varshney et al. predicts each model's correctness from a small encoder's prompt activations (Varshney et al., 2026), and pre-generation activations are known to encode whether a model will fail (Lugoloobi et al., 2026). These routers, like SWE-Router for agentic software engineering (Son et al., 2026), price each model at a per-model constant; the prefill router uses the median training output length, which is our main reference. Fewer routers predict per-query cost: MixLLM regresses output length on query embeddings (Wang et al., 2025), CARROT uses embedding nearest neighbours or a fine-tuned RoBERTa (Somerstep et al., 2025), GraphRouter predicts cost on a query–model graph (Feng et al., 2024), Route-To-Reason predicts reasoning-model length from text embeddings (Pan et al., 2025), and ZeroRouter prices queries by item-response difficulty bins (Yan et al., 2026). We compare against the embedding-based and ZeroRouter estimators directly, with every estimator given the same success predictions.

Cascades instead call models in sequence and stop when an answer is accepted: FrugalGPT learns a scorer that decides whether to escalate (Chen et al., 2023), AutoMix uses self-verification (Aggarwal et al., 2023), and Dekoninck et al. unify routing and cascading (Dekoninck et al., 2024). A cascade pays for the cheap attempt on every query; we route once, before generation. Activation-based length prediction also supports serving and scheduling of a single model (Messenger, 2026; Xie et al., 2026); we predict the length of other models from one shared prefill. Hosted open-weight models vary across providers in accuracy, length and price (Li et al., 2026; Lin et al., 2026; Gao et al., 2025), and FACET selects among providers of a fixed model per task from measured outcomes (He et al., 2026).

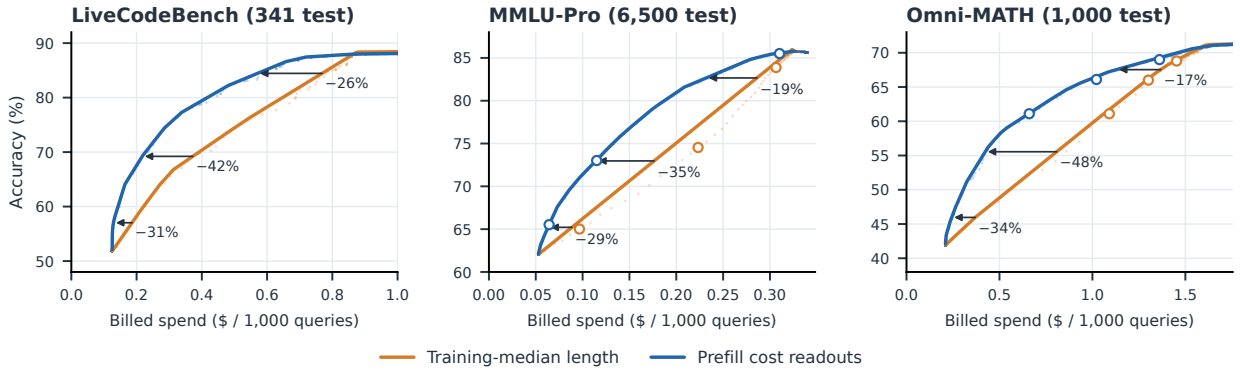


Figure 2: **Cost–accuracy frontiers at billed prices.** Prefill cost readouts (blue) versus training-median length (orange), with the same success predictions and the same encoder pass. Dots: the deterministic policy at each V; lines: their frontiers; hollow circles: policies selected on calibration for fixed accuracy targets. Arrows run from the median rule's frontier to ours at matched accuracy and give the cost saved; averaged over the shared accuracy band the saving is 34% on LiveCodeBench, 29% on MMLU-Pro and 35% on Omni-MATH.

2 Cost Prediction and Routing

For query x and route m (a model at a fixed reasoning effort, served by an endpoint), let $p_m(x)$ be the probability of a correct answer and $c_m(x)$ the expected generation cost. One route is chosen before generation,

$$m^*(x; V) = \arg\max_m \{V\hat{p}_m(x) - \hat{c}_m(x)\}, \tag{1}$$

where V is the value of a correct answer; sweeping V trades accuracy for cost. Every query is answered once, with no verifier or fallback.

Readouts. We concatenate mean and last-token activations from eight layers of a frozen Qwen3-4B (Instruct-2507; Thinking-2507 on Omni), standardized on training data. Success readouts are L2-regularized logistic regressions with held-out Platt calibration; cost readouts are ridge regressions of log mean output length, regularized by training-only cross-validation. Predicted length is exponentiated with residual smearing and matched to the route's training mean, then priced: $\hat{c}_m(x) = r_m^{\text{in}} n_m^{\text{in}}(x) + r_m^{\text{out}} \hat{\ell}_m(x)$.

3 Experimental Protocol

The routes are gpt-oss-20b at low and medium effort, deepseek-v4-flash, and gpt-oss-120b at medium and high effort, called through OpenRouter, with deepseek-v4-flash pinned to one provider (StreamLake) throughout, because its output length depends on the serving provider (see Endpoints). LiveCodeBench (Jain et al., 2024) has 892 problems split by date into 441 training, 110 calibration and 341 test problems. Omni-MATH (Gao et al., 2024) and MMLU-Pro (Wang et al., 2024) have 275/75 and 550/150 training/calibration problems and 1,000 and 6,500 disjoint test problems. Training problems have 2–16 draws per route; Omni-MATH and MMLU-Pro test problems have one. Table 2 gives each route's accuracy and cost.

Prices. Realized cost is the cost billed for each call (on LCB, realized tokens at the billed rates). Predictions are priced at each model's effective billed rate, fitted on the test calls (USD per million output tokens: 0.13 for gpt-oss-20b, 0.08 for pinned deepseek-v4-flash, 0.26 for gpt-oss-120b). These differ from the list prices by up to 2.3×.

Evaluation. All estimators are fitted on training problems only and evaluated on the test sets, with the same success predictions. Each V on a dense grid gives a deterministic policy and an observed (cost,

Cost saved by ours vs.	LCB (n=341)	Omni-MATH (n=1,000)	MMLU-Pro (n=6,500)
Training-median length	33.7 [24.1, 39.9]	34.8 [27.9, 39.8]	28.8 [26.0, 31.8]
Training-mean length	29.6 [23.6, 34.5]	35.8 [28.9, 40.5]	28.6 [25.9, 31.7]
Prompt-feature GBM	26.3 [16.3, 32.9]	26.5 [18.9, 31.9]	25.2 [21.0, 28.7]
MixLLM-style embeddings	18.2 [11.2, 25.2]	26.7 [20.0, 32.5]	19.2 [15.3, 22.5]
ZeroRouter (reimplemented)	21.1 [13.2, 26.8]	14.4 [8.4, 20.6]	29.0 [25.5, 32.0]
Headroom: oracle vs. median	45.3 [36.6, 50.4]	55.5 [50.4, 58.8]	59.5 [57.8, 61.2]

Table 1: **Cost saved at matched accuracy, billed costs.** Cost saved (%) by our prefill cost readouts relative to each estimator, with 95% paired bootstrap intervals; same success predictions for every arm except ZeroRouter, which uses its own; all intervals exclude zero. Last row: cost saved over median pricing by perfect per-query cost knowledge (each problem priced at its realized output), with the same success predictions; our readouts capture 74%, 63% and 49% of it. Test sets as in Fig. 2. Prompt-feature GBM: gradient boosting on prompt statistics, as in latency-aware routing (Patel et al., 2026). MixLLM-style: jina-embeddings 137M with the MixLLM ensemble head (LCB, code-aware variant). ZeroRouter: item-response latent fitted on the five routes, 4B features, its own success model and bin pricing, configuration chosen on calibration.

	LCB		Omni-MATH		MMLU-Pro	
Route	Acc.	Cost	Acc.	Cost	Acc.	Cost
gpt-oss-20b, low	51.8	0.13	41.9	0.21	62.1	0.05
gpt-oss-20b, medium	68.2	0.65	57.5	1.20	69.4	0.18
deepseek-v4-flash	89.3	0.90	71.3	1.66	86.2	0.33
gpt-oss-120b, medium	82.1	0.64	60.3	1.12	77.2	0.23
gpt-oss-120b, high	89.5	3.73	66.6	4.57	79.3	0.90

Table 2: **Routes on the test sets.** Accuracy (%) and mean billed cost per call (USD per 1,000 calls). No route dominates on every query: costs differ 17–29× across routes, and accuracy is not monotone in price.

accuracy) point. Comparing two cost estimators a, b, we interpolate each one's cost at 12 accuracies spanning the interior 90% of the range both reach and report $1 - \exp(\overline{\log C_a/C_b})$, the cost saved by a at matched accuracy. Intervals are percentile intervals from paired problem bootstraps (300 resamples), keeping each problem's routes together. As an upper bound, an oracle arm prices each problem at its realized cost; its saving over median pricing is the headroom, and our share of it the capture. To check that an operating point can be chosen in advance, we also select the cheapest V reaching each accuracy target on calibration and apply it once.

4 Results

Cost readouts beat every estimator. At matched accuracy, prefill cost readouts spend 33.7% less than median-length pricing on LCB, 34.8% less on Omni-MATH and 28.8% less on MMLU-Pro (Table 1). That is 74%, 63% and 49% of the headroom, the saving perfect per-query cost knowledge would bring (45–60%). Every external estimator loses with intervals excluding zero: mean length by 29–36 points, a prompt-feature model by 25–27, MixLLM-style embeddings by 18–27 and a full ZeroRouter reimplementation by 14–29.

Pool	Domain	Test n	Headroom	Ours vs. median	Capture	Ours vs. cost-from-success
LiveCodeBench	code	341	45.3	33.7 [24.1, 39.9]	.74	−0.8
Omni-MATH	math	1,000	55.5	34.8 [27.9, 39.8]	.63	−0.1
MMLU-Pro	knowledge	6,500	59.5	28.8 [26.0, 31.8]	.49	**24.1**
AIME 1983–2024	math	281	29.3	15.7 [11.7, 20.6]	.54	2.0
APPS	code	377	32.1	15.1 [5.0, 26.9]	.47	−3.7
BigCodeBench	code	421	12.6	6.3 [−1.6, 11.8]	.50	3.2
CodeContests	code	266	34.5	6.3 [−0.2, 17.0]	.18	0.7
SuperGPQA	knowledge	300	56.6	27.5 [10.9, 38.2]	.49	9.8
BIG-Bench Extra Hard	reasoning	300	41.0	28.6 [19.0, 36.7]	.70	**24.5**

Table 3: **Headroom and capture on every pool.** Cost saved (%) at matched accuracy relative to median-length pricing: by perfect per-query cost knowledge (headroom) and by our readouts (95% paired bootstrap intervals); capture is their ratio. Last column: cost saved by our dedicated cost readout relative to our ablation that prices from the success readouts alone; bold: interval excludes zero. Billed costs, deepseek-v4-flash pinned, same success predictions in every column.

Choosing V in advance costs little: policies selected on calibration spend 1.0–4.7% more than our own test frontier at the accuracy they reach on the six pools where we save most (hollow circles in Fig. 2; Section 11).

Endpoints. The same model served by different providers behaves like the model plus an offset: pinning deepseek-v4-flash to each of three providers, they agree on correctness for 90–96% of problems and their output lengths correlate at .83–.95, but one writes about half as many tokens as another. Reusing the readouts with one multiplicative length offset per endpoint, estimated from 50 calls, removes the resulting cost bias on average, though a single estimate is noisy. A few dozen calls per provider also identify the cheapest one, and pinning it beats routing over providers; we therefore pin deepseek-v4-flash throughout, which also makes its length more predictable and raises our saving on every large set.

Other pools. On the remaining six pools (Table 3) our readouts save 15.7% [11.7, 20.6] on AIME, 15.1% [5.0, 26.9] on APPS, 27.5% [10.9, 38.2] on SuperGPQA and 28.6% [19.0, 36.7] on BIG-Bench Extra Hard relative to median pricing; Section 8 treats BigCodeBench and CodeContests, where no estimator saves significantly. These numbers are not an artefact of one split: over ten random re-splits of each pool with the original split sizes, refitting every readout and estimator, our saving relative to median pricing is 34.1% ± 3.5 on LCB, 25.8% ± 2.3 on MMLU-Pro, 32.1% ± 5.8 on SuperGPQA and 24.8% ± 5.7 on BIG-Bench Extra Hard, and in none of the 90 splits is any external estimator significantly cheaper than ours (Table 11).

5 How much is per-query cost worth?

A cost estimator can be no better than knowing each query's realized cost. We therefore measure, on every pool, the *headroom*: the cost saved at matched accuracy when each test problem is priced at its realized cost instead of the training median, with the same success predictions. Table 3 reports it with our saving and the share of it we capture, on nine pools under one protocol (five routes, deepseek-v4-flash pinned, billed rates, test splits).

Headroom is large wherever reasoning lengths vary. On seven of nine pools perfect per-query cost knowledge would save 29–60%, and our readouts capture about half of it or more. The exceptions are informative. BigCodeBench has little to capture (12.6%), and CodeContests has headroom that our readouts do not reach (capture .18); Section 8 shows that no estimator reaches it.

Length, not only price gaps, makes it pay. To separate the two sources of headroom, we rescale every route's billed price toward or away from the cheapest route's while keeping realized lengths fixed. With all routes at the same per-token price, our readouts still save 13.3% (LCB), 14.3% (Omni-MATH) and 36.3% (MMLU-Pro) relative to median pricing: per-query length alone is worth that much. On LCB and Omni-MATH the saving grows with the price gap and saturates near the billed gap (3.1×); on MMLU-Pro it is

	Headroom		Ours vs. median		Capture	
Test set	reasoning	non-reas.	reasoning	non-reas.	reasoning	non-reas.
LiveCodeBench (341)	45.3	28.9	33.7	19.7 [7.7, 29.2]	.74	.68
Omni-MATH (1,000)	55.5	40.7	34.8	7.2 [3.7, 10.8]	.63	.18
MMLU-Pro (2,000)	60.5	26.0	30.6	1.3 [−2.4, 5.3]	.51	.05

Table 4: **Reasoning versus non-reasoning routes on the same problems.** Headroom (perfect per-query cost knowledge vs. median pricing), our saving relative to median pricing (95% paired bootstrap intervals) and capture, for the five reasoning routes and for five non-reasoning routes, on the test problems where every route of both pools has a valid answer (MMLU-Pro: a 2,000-problem subset of the test set). Billed costs; readouts fitted on the same training problems with the same recipes. Qwen3-235B-A22B-Instruct has training answers for 241 of 275 Omni-MATH and 485 of 550 MMLU-Pro problems (rate limits).

largest at flat prices (36.3%, against 28.8% at billed prices), because there length differs between questions rather than between routes. Widening the gaps beyond the billed ones shrinks the headroom (Fig. 5 in the appendix): at a 29× spread between the most and least expensive route it is 7–49% across pools, at 87× 2–42%, and our saving falls with it (1–25% at 87×). When prices differ by two orders of magnitude, choosing the model dominates and per-query length matters little, as for the software-engineering agents of Section 9.

Choosing effort and choosing the model are both needed. Restricting the pool removes most of the saving. With only gpt-oss-20b's two efforts, ours saves 21.2% (LCB), 21.8% (Omni-MATH) and 7.9% (MMLU-Pro); with only gpt-oss-120b's, −1.8%, 12.6% and 8.6%; with one effort per model, 12.8% and 17.8% on LCB and Omni-MATH; with all five routes, 33.7%, 34.8% and 28.8%.

Reasoning makes per-query cost worth predicting. To test whether this is a property of reasoning models, we collected five non-reasoning routes on the same problems: deepseek-v4-flash with thinking disabled (same provider and sampling), Llama-3.1-8B, Qwen3-30B-A3B-Instruct, Llama-3.3-70B and Qwen3-235B-A22B-Instruct, each pinned to one provider, one draw per problem, with output capped at 16k tokens. Readouts are fitted exactly as for the reasoning routes. Table 4 compares the two pools on the common test problems. Headroom is 1.4–2.3× larger with reasoning routes, and our saving is 1.7× (LCB) to about 24× (MMLU-Pro) larger; on MMLU-Pro the non-reasoning saving is not distinguishable from zero. Non-reasoning lengths do vary (s.d. of log length .53–1.73 per route), but the variation has two sources that the prompt predicts poorly. Llama-3.1-8B, at its default sampling settings, falls into repetition until the 16k cap on 9–12% of math problems (four in five capped answers loop), so its mean output is 4–6× its median and its log-length R^2 is below zero on every set. This is the model rather than the provider: redrawn on 300 such problems, it loops again on 29% with the same provider and 19% with another, against about 5% on control problems for both; a repetition penalty of 1.1 removes most loops (6%), apparently at some cost in accuracy. Qwen3-30B-A3B-Instruct, asked for code only on LCB, reasons inside code comments on hard problems until the cap (9% of problems), so its lengths span 142× between its 10th- and 90th-percentile problems (R^2 .27): the largest non-reasoning headroom comes from a non-reasoning model that reasons anyway. Adding the non-reasoning routes to the reasoning pool lowers our saving (17.7% on Omni-MATH, 13.0% on MMLU-Pro, 24.9% on LCB). The same model with thinking on and off shows the trade directly: deepseek-v4-flash with thinking gains 2.8 (MMLU-Pro), 14.6 (Omni-MATH) and 22.1 (LCB) points of accuracy for 13×, 11× and 33× the output, and the problems that are long with thinking are long without it (correlation of log length .66–.76).

6 What the cost readout captures

Length is mostly a property of the problem. The prefill explains much of each route's log output length on held-out problems (5-fold cross-validated R^2: .73–.77 on LCB, .74–.83 on Omni-MATH, .52–.66 on MMLU-Pro, .43–.57 on AIME). On LCB, Omni-MATH and AIME most of this is shared with difficulty: a problem's empirical solve rate across routes alone reaches R^2 .48–.65, and the prefill misses at most .07 of what difficulty explains (.10–.22 on AIME). Within a problem, failed and solved draws are about equally long

(×0.83–1.16), with one exception: deepseek-v4-flash writes 2.5× (Omni-MATH) and 1.9× (AIME) longer when it fails on hard math.

The shared level carries most of the value. A problem's lengths across routes can be split into a shared level (their mean) and route-specific differences. On LCB, an oracle that knows only each problem's level captures 87% of the headroom, about as much as one that knows only the differences. The prefill reads the level far better (R^2 .75) than the differences (.41); pricing with the predicted level alone keeps 33.2 of the full readout's 36.4 points (list prices).

Success and cost are read from the same layers. Refitting both readouts on each stored layer separately, success AUC and length R^2 peak in the same late layers at the last token (layers 22–36 of 36), and one such layer matches the full concatenation (LCB AUC .86–.87, R^2 .73–.75, saving 33–34%; Omni-MATH .83–.84, .71–.72, 35–37%; MMLU-Pro .72, .48–.50, 27–29%). The two readouts are close to one signal where length tracks difficulty: the mean success logit correlates with predicted log length at $-.90$ to $-.97$ per layer on LCB and $-.71$ to $-.84$ on Omni-MATH, but only $-.36$ to $-.58$ on MMLU-Pro.

Where length tracks work rather than difficulty. On MMLU-Pro, difficulty explains log length only in part (test R^2 .18–.37), subject labels about as much (.17–.25), and the two together .34–.44, against .41–.66 for the prefill. Engineering and law questions are both long (6.4k and 4.1k output tokens on average) and hard. Pricing from difficulty and subject together saves 30.3%, against 43.9% for the prefill (headroom 64.1%, list prices): the prefill reads how much work a question requires, beyond its difficulty and topic.

The success readouts carry more than difficulty. Even where pricing from success ties the dedicated readout, it does not price by difficulty alone. A problem's true solve rate over the five routes (with its square) is a weak price even in-domain: it saves 21.5% on LCB against 34.4% for pricing from the success logits, and nothing on the 500-problem Omni-MATH pool (-4.1% against 18.0%), because the logits carry length cues beyond the solve rate. The part of the predicted length that the true solve rates do not explain is real. It correlates with the same part of the realized length at .56–.92 across six pools (MMLU-Pro .81: engineering questions are predicted 0.95 log units longer than their difficulty implies, and are 0.95 longer). It also predicts the length of five held-out non-reasoning models on MMLU-Pro (partial correlation .40–.71) while saying almost nothing about whether the stronger of them succeed ($|r| \leq .05$ for deepseek-v4-flash without thinking and Qwen3-235B-Instruct). We read it as the work a problem requires, which difficulty proxies only in part.

Two further heterogeneous pools. We chose SuperGPQA (M-A-P Team, 2025) and BIG-Bench Extra Hard (Kazemi et al., 2025) as further pools because, on a cheap screen (one gpt-oss-20b-low draw per problem), length predicted directly from the prefill beat length predicted from a success readout by a margin as large as MMLU-Pro's (R^2 gap .61 and .40, against .66 on MMLU-Pro and .10 on Omni-MATH). On their full pools the dedicated readout again beats pricing from success on BIG-Bench Extra Hard, by 24.5% [15.5, 33.3], where pricing from success saves only 5.5% over the median; on SuperGPQA the margin is smaller and its interval includes zero (9.8% [−0.7, 17.5]; Table 3), so the screen overstated the separation there. On BIG-Bench Extra Hard, text-only estimators read length as well as the prefill: MixLLM-style embeddings and prompt features both save 28.6–28.7% relative to median pricing (ours −0.4 [−4.1, 4.7] and −0.3 [−4.9, 4.0] relative to them). Its 23 tasks have very different formats, and the task is visible in the text. On SuperGPQA they do not (ours saves 15.0% [−1.3, 25.3] relative to MixLLM-style and 18.8% [5.5, 29.6] relative to prompt features).

How many length labels are needed. Given success readouts, where length tracks difficulty a cost readout needs few length labels (Fig. 3). Pricing from the success logits reaches 27.8% (LCB) and 31.5% (Omni-MATH) with 20 labelled training problems, against 24.7% and 22.8% for a dedicated readout. On MMLU-Pro it plateaus at about 6%, while the dedicated readout climbs to 22.6% at 100 problems and 28.8% at all 550. One draw per training problem instead of all draws gives up 1–3 points.

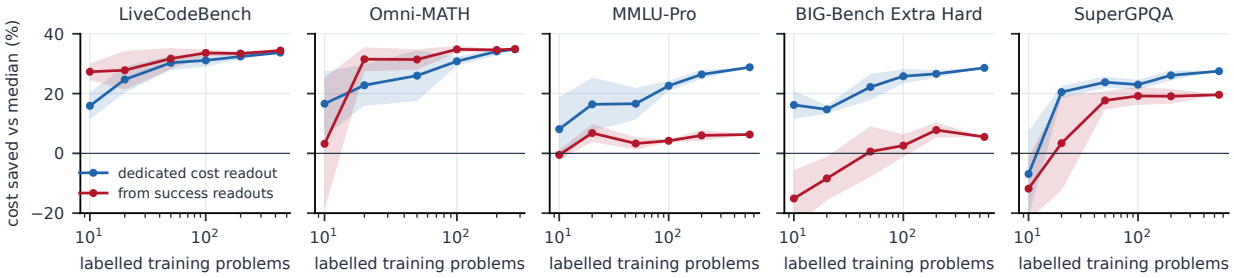


Figure 3: **Length labels needed.** Cost saved relative to median pricing when the cost readout is fitted on n labelled training problems: a dedicated readout on the prefill (blue) versus pricing from the success readouts (red), which needs length labels only to map success logits to length. Mean over five random subsets; bands ±1 s.d. Where length tracks difficulty (LiveCodeBench, Omni-MATH), pricing from success is as good from 20 problems on; where it tracks the work a question requires (MMLU-Pro, BIG-Bench Extra Hard), it stays below 8% at every n.

What changes in the routing. At matched accuracy in the middle of the band, 31–66% of problems change route relative to median pricing. The dominant changes are between gpt-oss-20b-low and deepseek-v4-flash: problems predicted to be short move to the more accurate deepseek-v4-flash (19–38% of problems), and problems predicted to be long move to the cheap route (9–18%).

7 How cheaply can a router get it?

Where length tracks difficulty, pricing from the success readouts ties the dedicated cost readout when both are fitted on a benchmark's full training set (Table 3). A deployed router rarely has that: it has few labels, it adds models, and it meets benchmarks it was not trained on. We test the cost estimators in all three settings, with every estimator given the same success predictions unless stated, and the same protocol as before.

Few labels. Table 5 refits every cost estimator on 20 randomly chosen training problems (five subsets). Our readout keeps 51–75% of its full-data saving on every pool (LCB 24.7%, Omni-MATH 22.8%, MMLU-Pro 16.4%, SuperGPQA 20.5%, BIG-Bench Extra Hard 14.7%), while the external estimators save less than 11% on every pool. Their deficit narrows with data but stays large on most pools: at 100 problems ours saves 22.6% on MMLU-Pro against at most 10.6% for any external estimator, with every paired interval excluding zero. The exception is BIG-Bench Extra Hard, where the task format is visible in the text: the text estimators reach 20–21% at 100 problems and tie ours at full data, though below 50 problems ours still leads (by 22.7 [5.8, 42.2] points over MixLLM-style embeddings at 50). Pricing from the success readouts is the strongest arm on LCB and Omni-MATH from 20 problems on, as in Fig. 3, and far behind ours on the two heterogeneous pools.

New models. To add a model, a router needs its success and cost readouts. We hold out one model at a time from the pool (both efforts of gpt-oss-20b, both efforts of gpt-oss-120b, or deepseek-v4-flash, so that no sibling effort of the new model remains) and fit its readouts from k labelled problems while every other route keeps its full readouts. Our onboarding fits two parameters per readout: an offset to the other routes' predicted log length, and a logistic map from their mean success logit. ZeroRouter's onboarding, its main use case, fits the new model's ability on the same k anchors in an item-response latent fitted on the remaining routes and fills its per-bin length table from them (Yan et al., 2026); a CARROT-style arm averages the k anchors' outcomes and lengths over nearest neighbours. Table 6 reports the mean over the three held-out models. From 10 examples our router saves 17.1–25.6%, against 4.4–22.4% for ZeroRouter's onboarding, and leads it by 10.1 [1.4, 23.7] points on MMLU-Pro and 15.7 [0.4, 31.5] on SuperGPQA; at k=50 it also leads on LCB (9.3 [3.0, 17.0]). On Omni-MATH and BIG-Bench Extra Hard the two are within noise. The gain concentrates on deepseek-v4-flash, the one model whose family is not otherwise in the pool, and whose

Cost estimator, 20 training problems	LCB	Omni-MATH	MMLU-Pro	SuperGPQA	BBEH
Ours (prefill readout)	**24.7**	22.8	**16.4**	20.5	14.7
Cost from success (our ablation)	27.8	31.5	6.8	3.4	−8.4
Prompt-feature GBM	−7.7	0.1	2.8	2.5	7.9
MixLLM-style embeddings	−1.8	7.0	2.5	3.4	−11.6
CARROT-style kNN	0.1	6.0	10.7	7.3	−5.7
ZeroRouter pricing	7.6	9.3	3.5	−9.5	−3.9
Ours, all training problems (refit)	33.7	34.8	28.8	27.5	28.6

Table 5: **Few labels.** Cost saved (%) relative to median pricing (from the full training set) when each cost estimator is fitted on 20 random training problems; mean of five subsets; every arm routes with the same success readouts. ZeroRouter pricing: its item-response latent fitted on the 20 problems, read from the prefill, with its bin lookup; kNN neighbours and ZeroRouter's dimension and bin count chosen on the calibration split. Bold: ours exceeds every external estimator with paired intervals excluding zero (LCB from n=20, MMLU-Pro from n=100); on SuperGPQA it does so at n=50 only, and on BIG-Bench Extra Hard the prompt-feature GBM stays within noise at every n.

	LCB	Omni-MATH	MMLU-Pro	SuperGPQA	BBEH
Ours, k=10	24.6	25.6	22.6	20.1	17.1
ZeroRouter onboarding, k=10	13.6	22.4	12.6	4.4	9.7
CARROT-style kNN, k=10	13.3	15.3	8.5	7.0	13.9
Median length + base rate, k=10	−7.1	10.6	2.9	3.3	10.3
Ours − ZeroRouter, k=10	10.9 [−2.0, 24.1]	3.2 [−12.3, 27.9]	10.1 [1.4, 23.7]	15.7 [0.4, 31.5]	7.4 [−1.9, 20.9]
Ours − ZeroRouter, k=50	9.3 [3.0, 17.0]	−0.2 [−7.3, 6.0]	9.6 [2.2, 14.8]	13.4 [5.8, 23.8]	3.7 [−1.1, 7.8]
All routes on full readouts	33.7	34.8	28.8	27.5	28.6

Table 6: **Onboarding a new model from k examples.** Cost saved (%) relative to the full pool at median pricing, mean over three held-out models (gpt-oss-20b with both efforts, gpt-oss-120b with both efforts, deepseek-v4-flash) and 20 random draws of the k anchors (the same anchors in every arm); every other route keeps its full readouts. Differences in points with 95% paired bootstrap intervals over test problems. ZeroRouter's dimension and bin count and the kNN neighbour count are chosen on calibration.

absence makes the router spend more than median pricing on four of five pools: onboarding it from 10 problems recovers 11–38 points more than ZeroRouter's onboarding does. ZeroRouter's Fisher-information anchors help both methods about equally; with the same anchors ours ties on LCB and Omni-MATH and leads by 5–14 points on the other three pools at k=10.

Three further model families. We added Qwen3-32B, GLM-4.7-Flash and Nemotron-3-Super-120B, all reasoning, on LCB, Omni-MATH and a 2,700-problem subset of MMLU-Pro (billed \$0.40–0.57 per million output tokens; Qwen3-32B's provider stops at 24,575 output tokens, and Nemotron has no valid answer for 27% of Omni-MATH problems after retries), and fitted the same readouts for them (test success AUC .72–.87, log-length R^2 .23–.81). With all eight routes our readouts save 34.8%, 33.0% and 28.6% relative to median pricing, beat difficulty bins on Omni-MATH (8.7 [2.1, 14.2]) and MMLU-Pro (21.3 [14.9, 26.0]) and pricing from success on MMLU-Pro (10.4 [5.1, 14.9]), and tie both elsewhere, as with five routes. In the full pool the new models are never needed (removing any one changes the saving by under 0.2 points), so we test onboarding in a pool of the cheap routes: both gpt-oss-20b efforts and the three new models. There only Nemotron matters (without it the saving falls from 36.8% to 26.1% on LCB and from 30.5% to 26.4% on MMLU-Pro). Onboarded from 10 problems, our router saves 26.0% on both, while ZeroRouter's onboarding saves −30.2% and 10.8%; at k=50 ours leads it by 34.9 [18.0, 61.3] and 18.6 [4.7, 35.6] points (Omni-MATH: 7.3 [−1.9, 23.5] at k=10). A badly onboarded model can cost more than it adds: from 10 examples, onboarding at median length and base rate leaves the router spending more than median pricing for seven of the nine model–pool pairs, ZeroRouter's onboarding for two, ours for none. Finally, the new models' readouts trained on Omni-MATH and used on MMLU-Pro keep the full saving (28.5% against 28.6% in-domain).

Target	Ours, in-domain	Ours, transferred	Prefill-router rule, transferred	Best external, transferred
LiveCodeBench	33.7	32.1	2.0	7.8 (MixLLM)
APPS	12.8	21.7	−0.7	11.0 (MixLLM)
MMLU-Pro	28.8	27.3	−1.5	17.3 (kNN)
AIME	15.7	14.1	−1.6	6.8 (kNN)
SuperGPQA	27.5	19.3	−22.5	4.0 (MixLLM)
BIG-Bench Extra Hard	28.6	8.6	−19.1	−5.4 (MixLLM)
Omni-MATH (500-problem pool)	18.2	7.8	−20.0	−0.5 (MixLLM)

Table 7: **A router moved to a new benchmark.** Cost saved (%) relative to the target's own median rule, at matched accuracy on its test set. Transferred: success and cost readouts trained on the other pools of the same domain (coding: LCB, APPS, BigCodeBench, CodeContests; math and knowledge: Omni-MATH, MMLU-Pro, AIME, SuperGPQA, BIG-Bench Extra Hard), with every choice made on their calibration splits. Prefill-router rule: the transferred success readouts with median pricing. External estimators transferred the same way, each with its own success model (MixLLM-style: logistic readout on the text embedding; kNN; ZeroRouter: its latent and bins). Ours exceeds every other arm with paired intervals excluding zero on LCB, MMLU-Pro, AIME and BIG-Bench Extra Hard. Omni-MATH uses its 500-problem pool (test 150), whose prefill matches the other pools'. BigCodeBench and CodeContests, where no in-domain estimator saves significantly, are omitted: there the transferred success readouts are badly miscalibrated (BigCodeBench log loss 1.17 against .67 in-domain) and the apparent gains are not robust.

New benchmarks. Finally, we move a whole router to a benchmark it has not seen: both readouts are trained on the other pools of the same domain (code, or math and knowledge), with every choice (regularisation, Platt calibration, neighbour counts, latent dimension, bin count) made on those pools' calibration splits, so no label of the target is used. Table 7 compares it with the router trained on the target itself. On LCB, MMLU-Pro and AIME the transferred router keeps 90–95% of its in-domain saving (32.1%, 27.3% and 14.1%), and on APPS it exceeds it. The prefill router's rule, the same transferred success readouts with median pricing, saves at most 2% on any of these, so the transferred cost readout carries the result, even though transferred success predictions are weaker (LCB AUC .65 against .83). Ours leads every external estimator transferred the same way, by 7–24 points on these four pools. Transfer is weaker on the two heterogeneous pools and on Omni-MATH (7.8–19.3%), but still ahead of every transferred external estimator, which save at most 4.0% there. Combining both sources of labels is better than either: with ten labelled target problems added to the source data, the cost readout saves 27.9% on MMLU-Pro and 13.1% on AIME (with the target's success readouts), against 8.1% and 4.4% from the ten problems alone, and 24.1% against 14.7% on BIG-Bench Extra Hard with twenty. Pricing from the success readouts transfers unevenly. With the target's own success readouts and a map trained on the other pools, it keeps its saving where the target's kind of difficulty resembles its sources' (APPS 17.1%, AIME 12.3%, CodeContests 9.3%) and collapses elsewhere: from 34.4% to −6.2% on LCB, 19.6% to −21.9% on SuperGPQA, 18.0% to 6.5% on Omni-MATH and 5.5% to −20.5% on BIG-Bench Extra Hard. Under the same conditions the dedicated readout saves 30.2%, 17.7%, 15.7% and 16.1% on those four, and ZeroRouter's difficulty-bin pricing transfers as unevenly as pricing from success.

Which estimators lose least. Holding the target's success readouts fixed, we bootstrap over test problems the drop from in-domain to transferred pricing for each estimator, and its difference from our drop. Ours loses significantly less than pricing from success on LCB (37.1 points [26.9, 46.0]), SuperGPQA (31.8 [16.3, 49.9]) and BIG-Bench Extra Hard (13.4 [1.4, 23.9]), and less than ZeroRouter's bin pricing on LCB (21.0 [11.7, 28.5]) and MMLU-Pro (4.0 [1.1, 8.2]); on the other pools the drops are within noise of each other, and on no pool does ours lose significantly more. Our own drop is significant only on the two heterogeneous pools (9.8 and 12.5 points).

Why the cost readout transfers. Training one readout on the target together with the other pools matches the target-only readout within 4.4 points on every pool, with no difference significant, so a single readout can serve all of them. Part of what transfers is shared structure: 24–40% of each route's variance in

log length lies between benchmarks, and the routes agree on which benchmarks are long (mean correlation of benchmark means across routes .84). Length R^2 under transfer is close to in-domain on LCB, APPS, AIME, Omni-MATH and SuperGPQA but falls sharply on MMLU-Pro (.02–.46 against .29–.63), where routing transfers fully nonetheless: routing needs the ordering of problems within a route and the contrast between routes, not calibrated absolute lengths. What does not transfer is the map from difficulty to length. Matching the problems of two benchmarks by solve rate (per route, five bins), their lengths still differ by 1.14 log units on average (×3.1 over 470 cells). Our cost readout, trained on neither benchmark, predicts these gaps (correlation .95, 90% of the squared gap), pricing from success 30%, and any price from difficulty alone none, by construction. The curve from solve rate to length also differs in shape: it is flat on BigCodeBench, and without BigCodeBench among LCB's sources a linear map from the mean success logit to length transfers to LCB (12.5% to 32.5%; in-domain 34.7%). Difficulty is a benchmark-specific proxy for work; the prefill readout reads cues of the work itself, which carry over when the sources contain the target's kinds of work.

8 When it works and when it does not

Per-query cost pays under two conditions: (a) output length varies within a route by more than prices differ between routes, which creates headroom, and (b) that variation can be read from the prompt, which lets an estimator capture it. Each of our two failures violates one condition, and neither is specific to our estimator (Fig. 4).

Little headroom: BigCodeBench. BigCodeBench tasks are library-usage functions with short, similar solutions; perfect cost knowledge saves only 12.6% [4.1, 19.0]. No estimator's saving relative to median pricing has an interval excluding zero: ours 6.3 [−1.6, 11.8], ZeroRouter 8.8 [−12.5, 19.6], MixLLM-style 2.8, cost-from-success 3.1, prompt features 0.2. Here length is the least tied to difficulty of any pool (the prefill-only share of length variance is largest on BigCodeBench), but there is little to gain from reading it.

Illegible headroom: CodeContests. CodeContests has as much headroom as APPS (34.5% [29.0, 42.0]), but no estimator captures more than about a quarter of it: ZeroRouter saves 9.3% [1.1, 20.0], difficulty bins 5.6% [1.2, 14.6], ours 6.3% [−0.2, 17.0], and the rest are within noise of zero or below it. Length again tracks difficulty, but difficulty itself is hard to read from these prompts: the prefill misses .11–.18 of the length R^2 that empirical difficulty explains, against at most .07 on LCB and Omni-MATH.

Non-reasoning routes. Non-reasoning routes combine both failures (Table 4): less headroom, and on Omni-MATH and MMLU-Pro a capture of .18 and .05, because much of their length variation is degenerate output. On LCB the readouts capture .68 of a smaller headroom.

TODO: RouterBench chat (little length variation) as a further low-headroom point, if rerun on this protocol.

9 Software engineering

We study software engineering in two regimes: one-shot patch generation, where per-query cost pays as on the coding pools, and agents, where the price gaps between models dominate.

One-shot patches. On 1,432 SWE-Smith instances (Yang et al., 2025), each of the five routes writes one patch (deepseek-v4-flash pinned as elsewhere), labelled by executing the repository's tests in a sandbox. Pass rates are low and close together (25–37%), but which instances are solvable is legible from the issue (success AUC .80–.83 per route on the 389 test instances where every route returned a patch). Output length varies 5–28× between a route's 10th- and 90th-percentile instances and is only modestly predictable (log-length R^2 .09–.37). Per-query cost knowledge is worth 25.2% [15.0, 31.1] over median pricing, and our readouts save 14.5% [5.8, 19.8], a capture of .58. They beat mean length by 7.8% [3.5, 13.2], pricing from success by 9.2% [2.2, 13.9], MixLLM-style embeddings by 6.8% [2.5, 10.6] and prompt features by 18.2% [8.1, 27.6]; difficulty bins (2.9% [−0.3, 5.7]) and ZeroRouter (1.7% [−4.7, 9.0]) are within noise. One-shot patch generation thus behaves like the coding pools.

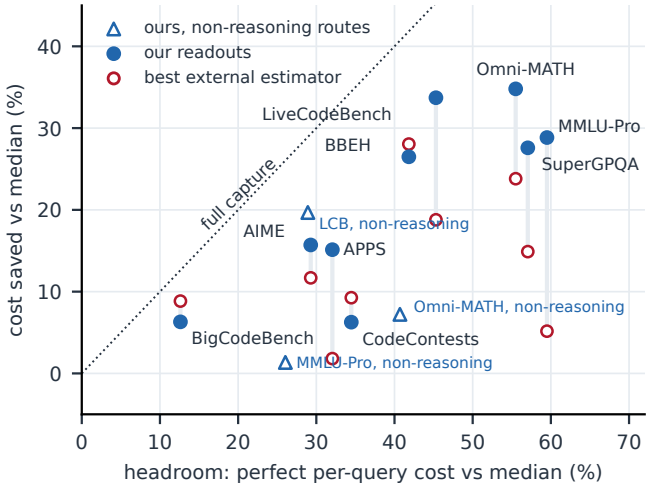


Figure 4: **Headroom and capture.** Per pool: the saving of perfect per-query cost knowledge over median pricing (x) against the saving of our readouts (filled) and of the best external estimator on that pool (hollow; mean length, prompt features, MixLLM-style embeddings or ZeroRouter). Triangles: our readouts with five non-reasoning routes on the same LiveCodeBench, Omni-MATH and MMLU-Pro test problems (Table 4). Billed costs, test splits, deepseek-v4-flash pinned. Where headroom is large and length is legible, the readouts capture half or more of it; on BigCodeBench there is little to capture, on CodeContests no estimator captures it, and non-reasoning routes leave less headroom, which on Omni-MATH and MMLU-Pro is almost all illegible.

Agents: cost is real, varies by task and is stable. Public SWE-rebench trajectories (Badertdinov et al., 2025) give 111 tasks run five times by each of 13 models in one scaffold. Across models, agentic cost is dominated by the model: of the variance of log cost, 74% is between models, 15% between tasks, 6% their interaction and 4% run to run. Within a model, however, per-task cost spans 2.7–9.0× between the 10th and 90th percentile, and it is stable across runs (intraclass correlation .69–.92). Whether knowing it pays depends on the price gaps between the routed models, as in Section 8: among all 13 models, whose prices per run span about 190×, per-task cost knowledge is worth 8.5% [−5.0, 17.8] (one-shot routing, success and cost from three runs, scored on the other two); among the seven open-weight models, whose prices span about 10×, it is worth 25.7% [6.3, 40.6], as much as on the reasoning pools.

Agents: cost is only partly legible before the run. From the issue text alone, the prefill's cost readout reaches out-of-fold R^2 of about zero on the 111 SWE-rebench tasks, too few to fit it. On 3,387 tasks with about 20 SWE-agent runs each (one model, Llama-3.1-70B; cost proxied by cumulative context and output length), the per-task mean cost is reliable (split-half reliability .91) and the issue text predicts it at R^2 .29, still rising with data (.18 with 100 tasks, .32 with 2,710). This is below what the prefill reaches on one-shot pools (.43–.83). Reading the first ten steps of a cheap model's trajectory predicts other models' per-task cost at R^2 .16 and the same run's final cost at .28, which suggests exploring cheaply before pricing the expensive candidates.

TODO: Decide whether the test-writer measurement (NEW_PATH 4.B.2) belongs here. Cite the nebius SWE-agent trajectories dataset (verify on its Hugging Face page).

10 Ablations

Is difficulty enough to price? Two ablations price each route from our own success readouts instead of a dedicated cost readout: ZeroRouter's bin lookup on mean success logit (ten quantile bins), and a ridge regression of log length on the success logits. Both come within a few points of the cost readout on LCB (ours saves 4.0% [−1.1, 9.0] and −0.8% [−5.6, 4.0]) and on Omni-MATH (6.4% [1.1, 11.6] and −0.1% [−4.6, 4.6]),

	Omni-MATH			MMLU-Pro		
Encoder	Ridge	kNN	MixLLM	Ridge	kNN	MixLLM
Qwen3-Embedding-8B	16.6	20.9	21.5	21.1	21.3	19.5
jina-embeddings 137M	23.8	25.3	26.6	17.8	20.4	18.9
MiniLM-L12	24.3	25.7	25.4	18.8	19.0	16.2

Table 8: **Representation and readout.** Cost saved (%) by our prefill readouts relative to each encoder–readout cell at matched accuracy on the Omni-MATH and MMLU-Pro test sets; billed costs, every encoder pass priced, same success predictions. Every paired bootstrap interval (200 resamples) excludes zero.

where output length tracks difficulty, and lose by 22.3% [18.8, 24.7] and 24.1% [21.1, 26.7] on MMLU-Pro. There, even empirical difficulty explains little of log length ($R^2 \approx .19$), while subject labels recover roughly 60% of the gap. The prefill encodes length information beyond difficulty, and that information pays where difficulty and length come apart.

Representation, readout and prefill size. Table 8 crosses three text encoders with three cost readouts: ridge, CARROT-style k-nearest neighbours and a MixLLM-style ensemble, each with our success predictions and with every encoder pass priced. Our readout saves 16–27% against every cell, including Qwen3-Embedding-8B, an encoder twice the prefill's size. On PCA-reduced 4B features the three readouts are within about 7 points of one another, so the gain comes from the representation rather than the regressor. Pricing the encoder pass changes no comparison by more than 3 points. Table 9 varies the prefill model while keeping prompts, layers and both readouts fixed. Every size saves 22–34% against median pricing, and length R^2 rises with size. None beats our prefill: Qwen3-1.7B and 8B come within 0.3–4 points (within the intervals on Omni-MATH), while 0.6B trails by 9–11. The result is not specific to Qwen: prefills from three other families (Phi-4-mini, Granite-3.3-2B, SmolLM2-1.7B) also save 25–31% against median pricing on both sets. Phi-4-mini matches our router on MMLU-Pro (-0.7 [-2.6, 1.3]) but trails by 10 points on Omni-MATH; Granite and SmolLM2 trail by 5–9.

Is the prefill router's success pipeline needed? The prefill router (Varshney et al., 2026) selects a layer, a pooling and a PCA dimension per target by cross-validation over the encoder's upper layers, concatenates the selected features and trains an ensemble of shared-trunk MLPs. We reimplemented this pipeline on the same encoder, splits and cost readout, and compared it with our per-route logistic readouts on fixed layers. The two are tied on all three test sets: ours saves -0.1% [-7.7, 4.0] on LCB, 1.1% [-1.0, 3.0] on MMLU-Pro and -2.2% [-5.7, 1.9] on Omni-MATH relative to it (success AUC .857 vs. .839, .715 vs. .714 and .829 vs. .842). On the two heterogeneous pools the comparison splits: ours saves 8.4% [-2.5, 18.4] relative to their pipeline on BIG-Bench Extra Hard, and their pipeline is ahead by 6.0% [0.4, 11.2] on SuperGPQA, where success is barely legible to either (AUC .58 and .57). Linear readouts on a fixed set of layers are enough on four of five sets, and they make the cost readout a drop-in addition: the same features feed both. They are also cheaper to fit: on 16 CPU threads, our success and cost readouts for five routes take about 3 minutes on LCB and MMLU-Pro (nine regularisation values per route, chosen on calibration), against about 7.4 minutes for the reimplemented pipeline (48 feature configurations $\times$ 5 folds per route, then ten networks); neither needs an encoder pass beyond the shared prefill, unlike the MixLLM-style and ZeroRouter estimators, which fit in seconds but read their own text encoder.

11 Deployment

Routing versus cascades. A cascade calls routes in sequence and stops when an answer is accepted (Chen et al., 2023; Aggarwal et al., 2023). To bound what any cascade could achieve on our pools, we give it a perfect judge: every ordered plan of the five routes is called cheapest first, each attempt is judged by its

	Omni-MATH			MMLU-Pro		
Prefill	R^2	vs. median	vs. ours	R^2	vs. median	vs. ours
Qwen3-0.6B	.62	31.9	−9.0	.33	22.0	−10.8
Qwen3-1.7B	.65	34.4	−2.8	.42	26.8	−3.0
Qwen3-4B (hybrid)	.67	29.3	−4.1	.45	25.8	−4.4
Qwen3-8B	.68	31.4	−0.3	.47	26.1	−4.1
Phi-4-mini (3.8B)	.65	31.4	−10.2	.45	28.4	−0.7
Granite-3.3 (2.5B)	.62	26.6	−9.4	.39	25.8	−4.7
SmolLM2 (1.7B)	.59	31.4	−7.6	.33	24.8	−4.9
Qwen3-4B-2507 (ours)	.70	34.8	–	.50	28.8	–

Table 9: **Prefill size.** Each prefill feeds both readouts, refitted with the paper's recipes on the training problems; Omni-MATH and MMLU-Pro test sets, billed costs. R^2: test log-length R^2 averaged over routes. vs. median: cost saved relative to median pricing with the same row's success predictions. vs. ours: cost saved by the row's router relative to ours (negative: it spends more); its paired intervals (300 resamples) include zero for Qwen3-1.7B and 8B on Omni-MATH and Phi-4-mini on MMLU-Pro.

true outcome at no cost, and the cascade stops at the first correct answer. A learned judge can only do worse. Even against this bound, one-shot routing with our readouts spends 29.7% [19.6, 37.7] less at matched accuracy on LCB and 31.9% [26.4, 37.3] less on Omni-MATH, because a cascade pays for every failed attempt before the one it accepts. On MMLU-Pro the two tie (−0.8% [−6.1, 3.9]): the cheapest route solves 62% of questions at about 6% of the most expensive route's cost, so failed first attempts are cheap.

Per-query budgets. Predicted costs also enforce per-query budgets. Under a hard cap on cost per query, where a call that would exceed the cap is cut off and fails, our router chooses the route most likely to succeed within the cap from the predicted length distribution. At billed prices on LCB, the constant-distribution rule needs 1.26× our budget to reach the same accuracy and the median rule 1.34× (geometric means over the accuracy band); on the smaller Omni-MATH and MMLU-Pro test splits (150 and 300 problems) the constant rule needs 1.31× and 1.18× and the median rule 1.08× and 1.06×.

Adding models and benchmarks. Section 7 covers the two maintenance operations a deployed router needs: onboarding a model from ten examples, and reusing a router trained on other benchmarks, alone or with a few target labels.

Deployable policies. Our main metric compares cost–accuracy frontiers on the test set; a deployed router must fix its trade-off V in advance. For each estimator and each of ten accuracy targets spanning the band every estimator reaches on the calibration split, we choose the cheapest V that reaches the target on calibration and apply it once to the test set. We compare its test cost with the median rule's deployed policies, interpolated at the accuracy it actually reaches (Table 10). Our deployed policies save 22–38% on LCB, Omni-MATH, MMLU-Pro, AIME, SuperGPQA and BIG-Bench Extra Hard, all with intervals excluding zero; they cost 1.0–4.7% more than our own test frontier at the accuracy they reach and miss their target by 0.6–8.5 points (AIME, with 139 calibration problems, misses most). They lead the best deployed external estimator on LCB, Omni-MATH, MMLU-Pro and SuperGPQA and tie it on AIME and BIG-Bench Extra Hard, and lead pricing from success on MMLU-Pro and BIG-Bench Extra Hard (SuperGPQA: 10.1 [−0.2, 19.0]). Deployed savings exceed the frontier savings mainly because median pricing deploys less well: its calibration-chosen policies fall 10–21% short of its own test frontier on MMLU-Pro, AIME, APPS and

Pool	Ours vs median, deployed	Best external (deployed)	Ours − best external	Ours − cost from success	Ours: frontier loss / target miss
LiveCodeBench	36.5 [27.2, 43.8]	ZeroRouter 26.9	9.1 [4.3, 15.5]	1.2 [−2.7, 3.9]	1.7 / 1.7
Omni-MATH	34.6 [30.3, 44.9]	*k*NN 19.1	17.6 [11.6, 26.4]	−1.1 [−3.9, 2.1]	2.1 / 3.4
MMLU-Pro	37.9 [34.4, 42.8]	MixLLM 21.2	16.1 [13.0, 20.3]	23.1 [20.1, 27.3]	1.0 / 0.6
AIME	22.2 [14.5, 28.8]	ZeroRouter 14.9	6.3 [−0.8, 12.5]	0.2 [−4.2, 4.4]	1.8 / 8.5
APPS	16.0 [−4.1, 33.1]	ZeroRouter 19.9	1.2 [−15.7, 11.6]	−5.5 [−15.2, 11.1]	10.1 / 3.2
BigCodeBench	7.8 [−3.8, 15.7]	GBM 0.6	6.4 [−3.5, 21.5]	4.2 [−10.2, 16.6]	9.7 / 5.5
CodeContests	21.0 [5.2, 30.9]	ZeroRouter 14.0	4.4 [−7.5, 15.0]	4.0 [−5.0, 11.6]	5.0 / 2.4
SuperGPQA	31.6 [8.6, 41.8]	MixLLM 12.7	18.9 [2.8, 36.8]	10.1 [−0.2, 19.0]	4.7 / 5.2
BIG-Bench Extra Hard	27.1 [19.6, 37.4]	GBM 29.1	−2.0 [−5.5, 1.4]	24.6 [14.6, 41.8]	3.3 / 2.2

Table 10: **Deployable policies.** Each estimator picks, on the calibration split, the cheapest V reaching each of ten accuracy targets, applied once to the test set (same success readouts for all). Cost saved (%) relative to the median rule's deployed policies at the accuracy reached, mean over targets, with 95% paired bootstrap intervals over test problems (V fixed). Last column: extra cost over our own test frontier at the accuracy reached (%) / mean |test accuracy − target| (points). Billed prices.

CodeContests, against at most 10.1% for ours. On APPS, BigCodeBench and CodeContests the intervals are wide.

The router's own cost. The features come from one forward pass of Qwen3-4B over the prompt: 127–1,216 tokens on average per pool, 19–28 ms median latency on one H100 at batch size one. A router that predicts success from the prefill already pays for this pass, so the cost readout adds nothing to it. Against no routing at all, the pass is not free when the routed models are as cheap as ours (well under a dollar per thousand queries). Self-hosted at an assumed $2 per H100-hour with unoptimised batched inference, it costs 2–10% of our deployed saving on Omni-MATH, AIME, MMLU-Pro and SuperGPQA and about 30% on BIG-Bench Extra Hard and APPS, whose prompts are long; at the input price of the smallest Qwen3 model listed on OpenRouter (14B, $0.12 per million tokens), it costs 6–51% of the saving, and more than the saving on APPS. Its share falls as the routed models get more expensive.

TODO: Provider endpoints in full (4.A.37–4.A.41, 4.A.55), or drop: the Results paragraph may suffice.

12 Pre-registered predictions

TODO: USER DECISION: keep or drop. Two-sided record: screen rule (4.A.9); GAIN confirmed on Omni-MATH and MMLU-Pro; APPS GAIN not confirmed unpinned (pinned 15.1% [5.0, 26.9]); AIME NO GAIN falsified (pinned 15.7% [11.7, 20.6]), because a single-draw cheapest-route screen understates predictability; separation screen called SuperGPQA and BIG-Bench Extra Hard (held on BBEH: +26.6 over cost-from-success; overstated on SuperGPQA: +9.0).

13 Discussion and Limitations

When does per-query cost pay? It pays when output length varies within a model more than prices differ between models: here lengths span 11–121× within a route (Fig. 1) while per-call costs span 17–29× across routes (Table 2), leaving 45–60% headroom over median pricing. It is captured when that variation is legible from the prompt. Where length tracks difficulty (LCB, Omni-MATH), even difficulty-only pricing captures most of it; where it tracks the work a question requires (MMLU-Pro), only a dedicated readout does, and that is also where we capture least (49%), leaving the most room for better cost predictors.

Generalisation. In-domain and with full training data, a router could price from difficulty and lose little on most pools. The case for a dedicated cost readout is that it holds up where a deployed router actually operates: with few labels, with new models and on new benchmarks, where both difficulty-only pricing and text estimators lose most of their saving (Section 7).

Limitations. Each route needs output-length labels, though a few dozen suffice for most pools. The main experiments use five routes from two model families; three further families were added on three sets (Section 7), and in the full pool the router needed none of them. Transfer is evaluated across benchmarks that share the routed models. Omni-MATH and MMLU-Pro test problems have one draw per route. Intervals are pointwise and assume independent problems, and Table 1 conditions on the fitted readouts; resampling the training problems as well and refitting every estimator widens them, but our saving over median pricing stays positive on both large test sets (Omni-MATH [23.2, 38.0], MMLU-Pro [22.9, 30.7]), and random re-splits give a spread of 2–6 points (Table 11). Only deepseek-v4-flash is pinned to a provider; the gpt-oss routes are not, since their provider changes price but barely length. The prefill comparison covers four model families at up to 8B. Billed rates were measured over one collection, and prices change within days. The ZeroRouter, MixLLM and CARROT baselines are reimplementations.

14 Conclusion

A frozen prefill that already predicts success also predicts cost. Per-query cost is worth predicting for reasoning models, whose lengths vary within a model by more than prices differ between models, and much less for non-reasoning ones. On held-out test problems at billed prices, prefill cost readouts reduce spending at matched accuracy and outperform every alternative estimator we tested; pricing from difficulty alone matches them only where output length tracks difficulty and only with full in-domain data. With few labels, for new models and on new benchmarks, the dedicated readout keeps most of its saving. Endpoints of one model share these readouts up to an offset, so the readouts can absorb provider drift from a few dozen calls, though pinning a provider is simpler.

References

Pranjal Aggarwal, Aman Madaan, Ankit Anand, Srividya Pranavi Potharaju, et al. AutoMix: Automatically Mixing Language Models, 2023. arXiv:2310.12963.

Ibragim Badertdinov, Alexander Golubev, Maksim Nekrashevich, et al. SWE-rebench: An automated pipeline for task collection and decontaminated evaluation of software engineering agents. *arXiv preprint arXiv:2505.20411*, 2025.

Lingjiao Chen, Matei Zaharia, and James Zou. FrugalGPT: How to Use Large Language Models While Reducing Cost and Improving Performance, 2023. arXiv:2305.05176.

Jasper Dekoninck, Maximilian Baader, and Martin Vechev. A Unified Approach to Routing and Cascading for LLMs, 2024. arXiv:2410.10347.

Tao Feng, Yanzhen Shen, and Jiaxuan You. GraphRouter: A Graph-based Router for LLM Selections, 2024. arXiv:2410.03834.

Bofei Gao, Feifan Song, Zhe Yang, et al. Omni-MATH: A Universal Olympiad Level Mathematic Benchmark For Large Language Models, 2024. URL `https://arxiv.org/abs/2410.07985`. arXiv:2410.07985.

Irena Gao, Percy Liang, and Carlos Guestrin. Model equality testing: Which model is this API serving? In *International Conference on Learning Representations*, 2025.

Liang He, Jingbo Wen, Yixiong Chen, Yue Yang, Qizhen Lan, Kangning Cui, and Xilu Wang. You cannot pick a provider from the price list: Market-aware routing for open-weight LLM inference, 2026.

Qitian Jason Hu, Jacob Bieker, Xiuyu Li, et al. RouterBench: A Benchmark for Multi-LLM Routing System, 2024. URL `https://arxiv.org/abs/2403.12031`. arXiv:2403.12031.

Naman Jain, King Han, Alex Gu, et al. LiveCodeBench: Holistic and Contamination Free Evaluation of Large Language Models for Code, 2024. URL `https://arxiv.org/abs/2403.07974`. arXiv:2403.07974.

Mehran Kazemi, Bahare Fatemi, Hritik Bansal, et al. BIG-Bench extra hard. *arXiv preprint arXiv:2502.19187*, 2025.

Haorui Li et al. When is the same model not the same service? a measurement study of hosted open-weight LLM APIs, 2026.

Guanjie Lin, Yinxin Wan, Shichao Pei, Ting Xu, Kuai Xu, and Guoliang Xue. Behavioral consistency and transparency analysis on large language model API gateways, 2026.

William Lugoloobi, Thomas Foster, William Bankes, and Chris Russell. LLMs Encode Their Failures: Predicting Success from Pre-Generation Activations, 2026. arXiv:2602.09924.

M-A-P Team. SuperGPQA: Scaling LLM evaluation across 285 graduate disciplines. *arXiv preprint arXiv:2502.14739*, 2025.

Glen Messenger. ALPS: Activation-Based Length Prediction for Intelligent LLM Inference Scheduling, 2026. URL `https://doi.org/10.5281/zenodo.19078431`. Zenodo. doi:10.5281/zenodo.19078431.

Isaac Ong, Amjad Almahairi, Vincent Wu, Wei-Lin Chiang, et al. RouteLLM: Learning to Route LLMs with Preference Data, 2024. arXiv:2406.18665.

Zhihong Pan, Kai Zhang, Yuze Zhao, and Yupeng Han. Route to Reason: Adaptive Routing for LLM and Reasoning Strategy Selection, 2025. arXiv:2505.19435.

Shivam Patel, Akaash R. Parthasarathy, Ankur Mallick, and Gauri Joshi. Beyond Accuracy and Cost: Latency-Aware LLM Query Routing for Dynamic Workloads, 2026. arXiv:2607.18253.

Seamus Somerstep, Felipe Maia Polo, Allysson Flavio Melo de Oliveira, et al. CARROT: A Cost Aware Rate Optimal Router, 2025. URL `https://arxiv.org/abs/2502.03261`. arXiv:2502.03261.

Seongho Son, Sangwoong Yoon, Jiahua Tang, Shuhan Wang, et al. SWE-Router: Routing in Multi-turn Agentic Software Engineering Tasks, 2026. arXiv:2607.00053.

Wei Song, Zhenya Huang, Cheng Cheng, Weibo Gao, et al. IRT-Router: Effective and Interpretable Multi-LLM Routing via Item Response Theory, 2025. arXiv:2506.01048.

Tanay Varshney, Annie Surla, Michelle Xu, et al. LLM Router: Rethinking Routing with Prefill Activations, 2026. URL `https://arxiv.org/abs/2603.20895`. arXiv:2603.20895.

Xinyuan Wang, Yanchi Liu, Wei Cheng, et al. MixLLM: Dynamic Routing in Mixed Large Language Models, 2025. URL `https://arxiv.org/abs/2502.18482`. arXiv:2502.18482.

Yubo Wang, Xueguang Ma, Ge Zhang, et al. MMLU-Pro: A More Robust and Challenging Multi-Task Language Understanding Benchmark, 2024. URL `https://arxiv.org/abs/2406.01574`. arXiv:2406.01574.

Huanyi Xie, Yubin Chen, Liangyu Wang, et al. Predicting LLM Output Length via Entropy-Guided Representations, 2026. URL `https://arxiv.org/abs/2602.11812`. arXiv:2602.11812.

Cheng Yan, Wuyang Zhang, Zhiyuan Ning, et al. Breaking Model Lock-in: Cost-Efficient Zero-Shot LLM Routing via a Universal Latent Space, 2026. URL `https://arxiv.org/abs/2601.06220`. arXiv:2601.06220.

John Yang, Kilian Lieret, Carlos E. Jimenez, et al. SWE-smith: Scaling data for software engineering agents. *arXiv preprint arXiv:2504.21798*, 2025.

A Pool statistics

TODO: Per pool: n, splits, draws per route, accuracy ladder, output p10/p50/p90, billed prices (pinned).

B Reimplemented baselines

TODO: ZeroRouter (choices, D/K sensitivity, own encoder, population curve; 4.A.30–4.A.36), MixLLM/CARROT-style, prompt GBM (cite latencyrouting).

C Random re-splits and price gaps

Pool	Headroom	Ours vs median	Ours − best external	Splits ahead of all external	Ours − cost from success	Splits ahead / behind
LiveCodeBench	45.9 ± 3.0	34.1 ± 3.5	12.9 ± 2.9	10	2.2 ± 2.6	4 / 0
Omni-MATH	41.7 ± 2.5	19.0 ± 3.3	12.0 ± 3.8	9	−1.4 ± 2.6	1 / 2
MMLU-Pro	58.6 ± 1.0	25.8 ± 2.3	16.3 ± 2.6	10	12.8 ± 6.1	10 / 0
AIME	28.5 ± 2.7	15.3 ± 2.6	9.5 ± 2.6	10	2.5 ± 2.4	3 / 0
APPS	39.5 ± 6.0	15.6 ± 4.9	3.9 ± 2.8	0	−1.5 ± 2.4	0 / 0
BigCodeBench	11.0 ± 6.2	3.6 ± 2.3	−1.8 ± 4.7	0	4.2 ± 2.7	1 / 0
CodeContests	34.8 ± 2.0	8.7 ± 4.5	2.1 ± 4.0	0	−0.1 ± 3.3	0 / 0
SuperGPQA	59.0 ± 2.8	32.1 ± 5.8	5.8 ± 5.0	3	10.4 ± 7.6	5 / 0
BIG-Bench Extra Hard	37.6 ± 3.5	24.8 ± 5.7	1.0 ± 1.1	1	20.6 ± 5.3	10 / 0

Table 11: **Ten random re-splits per pool** (original train and calibration sizes; success readouts refitted with the paper recipe, every cost estimator refitted, configurations chosen on calibration). Mean ± s.d. over splits of cost saved (%) relative to median pricing, and of our saving relative to the best external estimator in that split (mean length, prompt GBM, MixLLM-style, CARROT-style kNN, ZeroRouter) and to pricing from success. "Splits ahead": splits where our 95% paired interval excludes zero against every external estimator, and (last column) ahead of / behind pricing from success. No external estimator is significantly cheaper than ours in any split. LiveCodeBench's paper split is temporal; its re-splits are random. On Omni-MATH and MMLU-Pro the re-splits move single-draw evaluation problems into training, which lowers our saving relative to the paper split (training there has 2–4 draws per route).

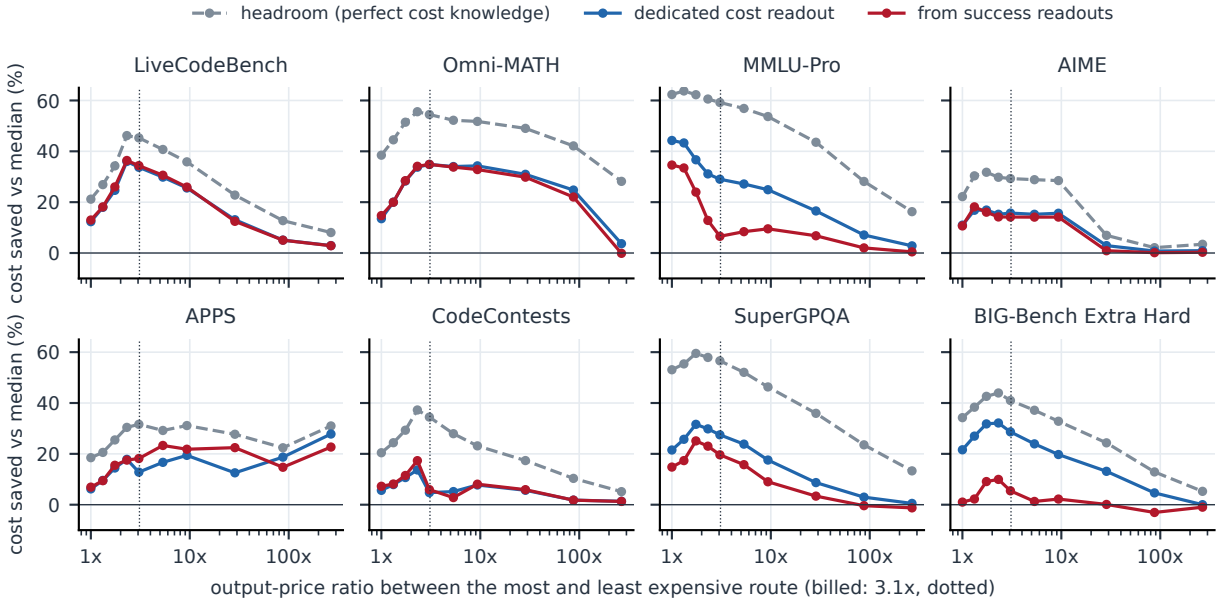


Figure 5: **Price gaps.** Cost saved relative to median pricing as every route's per-token price is moved toward or away from the geometric mean of the routes' prices (r_m^α, rescaled; α from 0, flat, to 5), with realized lengths fixed and costs recomputed from token counts (so values differ slightly from the billed-cost figures in the text). Dashed: headroom (perfect per-query cost knowledge); blue: our cost readout; red: pricing from the success readouts. Dotted: the billed spread (3.1×). BigCodeBench is omitted (its matched-accuracy band degenerates at some spreads).

D Protocol details

TODO: Frontiers/hulls, matched-accuracy band, paired and refit bootstraps, price fitting and cross-fitting, provider pinning.